

Metody Monte Carlo

Raport 3

Autor: Michał Frąckowiak

Dzisiejsza data

Spis treści

- 1 Problem 1 – Generowanie rozkładów warunkowych i wielowymiarowych 2
- 2 Problem 2 – Przybliżone obliczanie całek metodą Monte Carlo 3
- 3 Problem 3 – Twierdzenia graniczne dla ciągów *i.i.d.* 4

1 Problem 1 – Generowanie rozkładów warunkowych i wielowymiarowych

- Niech $X \sim \text{Bin}(1, p)$ i $Y | X = 0 \sim N(0, 1)$, $Y | X = 1 \sim N(\mu, \sigma)$, gdzie $p \in [0, 1]$, $\mu \in \mathbb{R}$, $\sigma > 0$. Zaimplementuj generator zmiennej Y . Uzasadnij, że $Y \sim (1-p)N(0, 1) + pN(\mu, \sigma)$. Narysuj histogram i wykresy funkcji gęstości tego rozkładu dla wybranych parametrów.
- Zaproponuj sposób generacji punktów z rozkładu jednostajnego na sympleksie $S := \mathbb{R}^2 \cap \{x, y > 0, x + y < 1\}$ oraz z rozkładu o gęstości cxy^2 na S .

$$\mu = (1, -1, 0) \text{ i } \Sigma = \begin{bmatrix} 2 & 2 & 0 \\ 2 & 5 & -3 \\ 0 & -3 & 9 \end{bmatrix}$$

- Zaimplementuj generator rozkładu $N(\mu, \Sigma)$, gdzie
Sprawdź, czy Σ jest symetryczna i dodatnio określona. Dla obliczenia $\Sigma^{1/2}$ użyj zaimplementowanej w R metody Choleskiego lub rozkładu spektralnego macierzy Σ . Wygeneruj próbkę z tego rozkładu i zilustruj wszystkie rozkłady 1-wymiarowe, 2-wymiarowe i 3-wymiarowe. Ile ich jest?

Rozwiązanie:

```

library(MASS) library(ggplot2)
set.seed(131311)

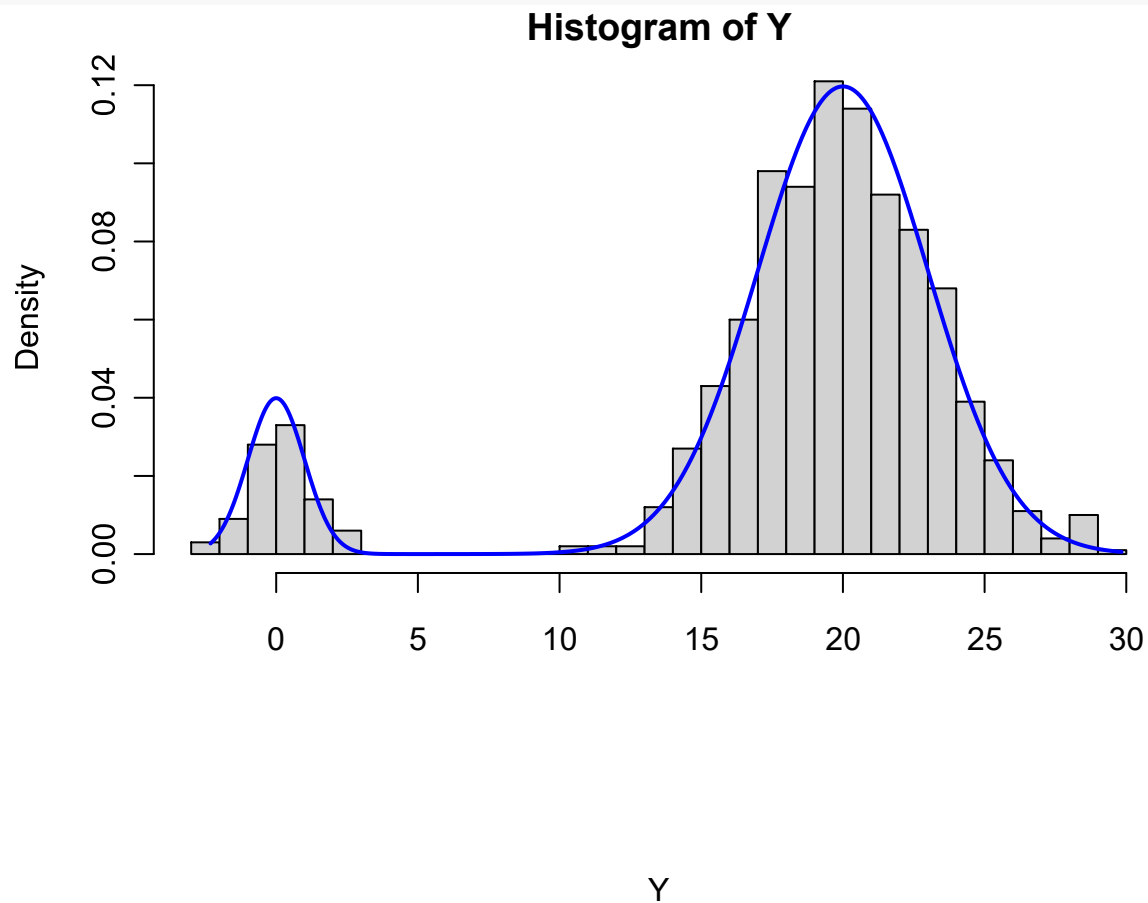
# 1.1
generate_Y <- function(p, mu, sigma, n) {
  X <- rbinom(n, 1, p)
  Y <- ifelse(X == 0, rnorm(sum(X == 0), 0, 1), rnorm(sum(X == 1), mu, sigma))
  return(Y)
}

p <- 0.9 mu
<- 20 sigma
<- 3 n <-
1000

Y <- generate_Y(p, mu, sigma, n) hist(Y, breaks =
30, probability = TRUE)

x <- seq(min(Y), max(Y), length.out = 1000) mix <- (1 - p) * dnorm(x, 0, 1)
+ p * dnorm(x, mu, sigma)
lines(x, mix, col = "blue", lwd = 2)

```

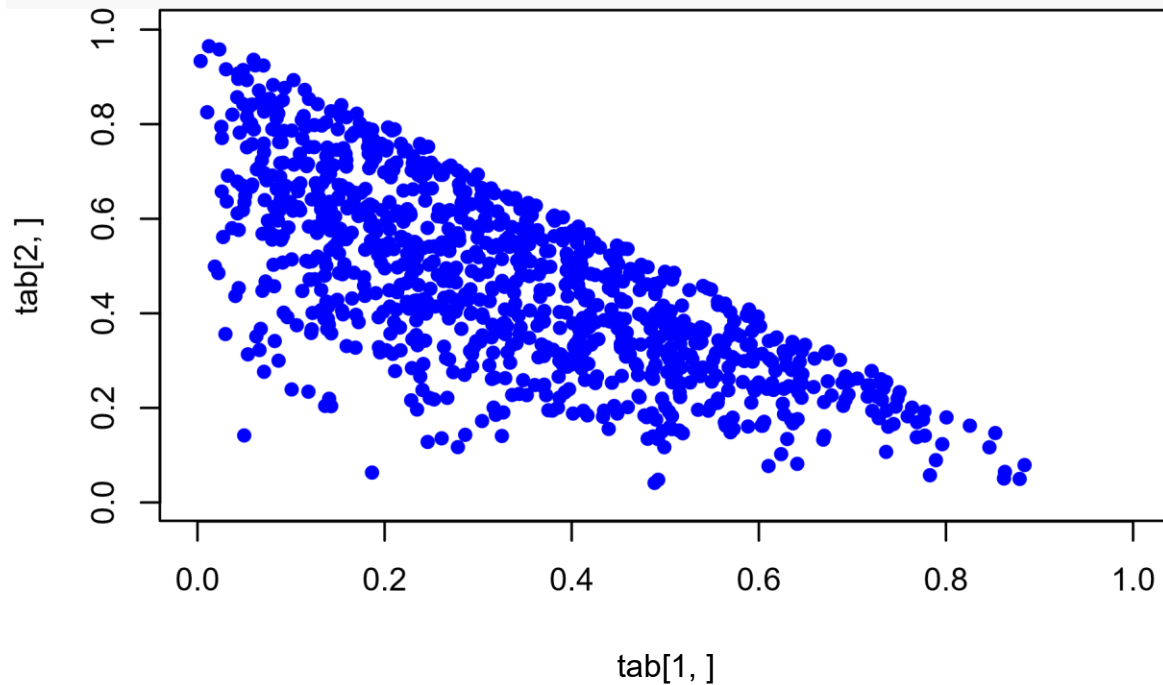


```
# 1.2 tab <- matrix(nrow = 2, ncol = 0)
while (length(tab) / 2 <= 1000) { x <-
runif(1, 0, 1) y <- runif(1, 0, 1)
```

```
  if (x + y >= 1) {
    next
  }

  gestosc <- 20 * x * y ^ 2

  if (gestosc > runif(1, 0, 1)) { tab <-
    cbind(tab, c(x, y)) }
} plot(tab[1,], tab[2,], col = "blue", pch = 16, xlim = c(0, 1), ylim = c(0, 1))
```



```
# 1.3
sigmq <- matrix(data = c(2, 2, 0, 2, 5, -3, 0, -3, 9), nrow = 3, byrow = TRUE) mi <- c(1, -1, 5) data <-
mvrnorm(n = 1000, mu = mi, Sigma = sigmq) print(all.equal(sigmq, t(sigmq)))
```

```
## [1] TRUE
```

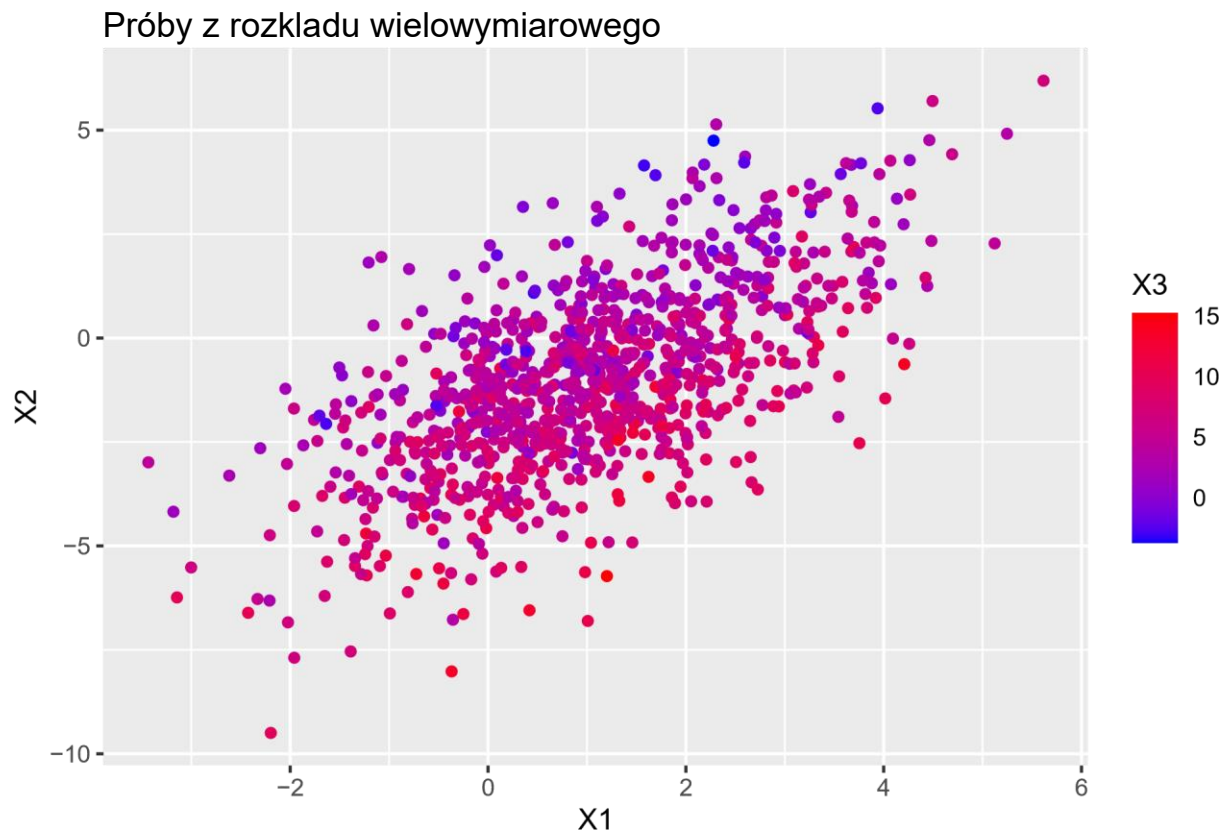
```
print(all(eigen(sigmq)$values > 0))
```

```
## [1] TRUE
```

```
cov_matrix <- matrix(c(1, 0.7, 0.7, 1), nrow = 2)
correlated_variables <- mvrnorm(n = 100, mu = c(0, 0), Sigma = cov_matrix)
```

```
df <- as.data.frame(data)
names(df) <- c("X1", "X2", "X3")
ggplot(df, aes(x = X1, y = X2, color = X3)) +
```

```
  geom_point() + scale_color_gradient(low = "blue", high = "red") + labs(title = "Próby z rozkładu wielowymiarowego",
  x = "X1", y = "X2", color = "X3")
```



1 Problem 2 – Przybliżone obliczanie całek metodą Monte Carlo

- Aproksymuj metodą Monte Carlo całkę $\int_0^1 e^{-x^2} dx$. Zbadaj empirycznie szybkość zbieżności i wariancję otrzymanego estymatora w zależności od rozmiaru próby. Znajdź przedział ufności dla tej całki na podstawie $n = 1000$ obserwacji.
- Użyj metody Monte Carlo do aproksymacji całki $\int_0^\infty x^5 e^{-x} \cos x dx$.
- Metodą Monte Carlo oszacuj powierzchnię żuka Mandelbrota. Jak ocenić błąd?

Rozwiązanie:

```
# 2.1 denisty_1 <- function(x) {  
  return(exp(1) ^ -(x ^ 2))  
}  
  
approximation <- function(f, n) { ext <- c(mean(f(seq(0, 1,  
  length.out = n))),  
  var(f(seq(0, 1, length.out = n))), sd(f(seq(0, 1, length.out =  
  n))))  
  return (ext)  
} print(approximation(denisty_1, 10))  
## [1] 0.73985415 0.05245709 0.22903512  
print(approximation(denisty_1, 100))  
## [1] 0.74618910 0.04145103 0.20359526  
print(approximation(denisty_1, 1000))  
## [1] 0.74676119 0.04050172 0.20125038
```

```

# Przedział ufności M <- 1000 n <- 100
alpha <- 0.1 tab <- vector("list", M) for
(j in 1:M) { U <- runif(n) p <-
c(sort(density_1(U))) tab[[j]] <- p
}
#print(tab)

temp <- c() interval <- c() for (i in 1:n) { for (j in
1:M) { temp <- c(temp, tab[[j]][i])
} temp <- sort(temp) interval <- c(interval, abs(temp[M - M * alpha] - temp[M *
alpha])) temp <- c()
} interval <- mean(interval)
print(interval)

```

```
## [1] 0.06735162
```

```

# 2.2 n <- 500000 U <- runif(n) density_2 <-
function(x) {
  return((exp(1) ^ (-x)) * (x^5) * cos(x))
} X <- density_2((1 / U) - 1) * (y ^ (-2))
print(mean(X))

```

```
## [1] -1.13939
```

```

# 2.3 rr <- 100
max_iter <- 1000 tt
<- 2
# Generowanie siatki punktów x <- seq(-2, 2,
length.out = rr) y <- seq(-2, 2, length.out = rr)
grid <- expand.grid(x = x, y = y) grid$c <-
grid$x + 1i * grid$y grid$in_set <- rep(TRUE,
nrow(grid))

# Funkcja sprawdzająca przynależność do zbioru Mandelbrota mandelbrot
<- function(c, max_iter, threshold) { z <- 0
  for (k in 1:max_iter) { z <- z ^ 2
    + c
    if (Mod(z) > threshold) {

```

```

        return(FALSE)
      }}
    return(TRUE)
  }

  # Sprawdzanie wszystkich punktów w siatce for (i in 1:nrow(grid)) {
    grid$in_set[i] <- mandelbrot(grid$c[i], max_iter, tt)
  }

  # Aproksymacja powierzchni area_approx <-
  mean(grid$in_set) * (4) * (4) print(area_approx)

```

```
## [1] 1.4624
```